07 / 24 — Phase 7 Assessment: Performance

This assessment covers lessons 01 through 23. It decides whether you are ready to move on from performance work to Phase 8 — Testing and quality and to the performance-sensitive parts of Phase 9 — Advanced rendering and ecosystem. It is not a recall quiz. Most items ask you to **predict, diagnose, implement, or decide**, because the real skill in this phase is choosing the *right* lever from evidence rather than reaching for a favourite one.

Recommended time	4–5 hours in one or two sittings
Format	Closed-book first pass (no editor, no docs, no profiler). Then optionally run code and traces to check yourself *before* reading the answer key
Total points	150
Pass	≥ 80% overall (120/150) **and** ≥ 70% in each of the four categories (see §7)

How to use it. Write answers on paper or in a text file. For code questions write code by hand, then run it. Do not read §6 (answer key) until you have attempted every part. Score yourself with §7, honestly: partial-credit rules are explicit.

Conventions. Unless stated otherwise: React 19.x with `createRoot`, TypeScript 5.x with `"strict": true`, **production builds**, Strict Mode off for render counts (Strict Mode double-invokes renders and effects in development only, see Phase 2 — Strict Mode behavior), "initial mount" is not counted in "re-renders" unless a question says so, and "the main thread" means the document's single main thread. Web Vitals thresholds quoted in this assessment are **as understood at writing; verify against https://web.dev/vitals/** before reusing them anywhere that matters.

Contents

1. Part A — Conceptual questions (40 pts)
2. Part B — Code and trace prediction (30 pts)
3. Part C — Debugging challenges (30 pts)
4. Part D — Implementation exercises (30 pts)
5. Part E — Architecture and engineering decisions (20 pts)
6. Answer key
7. Scoring rubric
8. Promotion criteria and remediation

Part A — Conceptual questions (40 pts)

20 questions, 2 points each. Answer in 2–5 sentences. The lesson each question targets is in brackets.

A1 [01]. Your colleague says "I profiled it on my MacBook with `npm run dev` and the component renders in 6 ms, so it isn't slow." List four reasons this conclusion is unreliable and say what a defensible measurement setup looks like.

A2 [02]. What does the React DevTools Profiler measure, and name three things that can make an interaction slow that it cannot see?

A3 [03]. State the three ways a component can be asked to render again, and explain why "its props did not change" is not, by itself, a reason a child skips rendering.

A4 [04]. Under what conditions does `memo` actually avoid work, and what does it cost on every parent render where it *fails* to avoid work?

A5 [05]. Give three kinds of prop values that are new on every render of the parent, and two different ways to make a `memo` child receive a stable value for each.

A6 [06]. A context value changes once per second and 300 components read *other* fields of the same value object. What happens on each change, why does wrapping consumers in `memo` not help, and name two structural remedies.

A7 [07, 08]. Explain *state colocation* as a performance technique (and the `children` composition variant of it), and name three costs that dominate a list of 5,000 rows that re-renders on every keystroke.

A8 [09]. How does windowing work mechanically, and name three costs or limitations that make it a poor default for a 100-row list.

A9 [10]. Order these by what you try first when a derived-data calculation shows 150 ms in a profile: `useMemo`, a Web Worker, an algorithmic or data-structure change, computing it once at update time instead of render time. Justify the order.

A10 [11, 12]. What does `useTransition` guarantee and what does it explicitly *not* do? Why must the text input's own state stay out of the transition?

A11 [13]. Describe the two-pass behavior of `useDeferredValue` and why the slow consumer usually needs `memo`.

A12 [14]. Distinguish debounce from throttle, name a good use case for each, and explain why debouncing a slow *render* is usually the wrong fix.

A13 [15, 16]. When is `React.lazy` + `Suspense` a good idea and when does it backfire? Name two things you look at in a bundle analysis and two things that commonly defeat tree shaking.

A14 [17]. Define a network waterfall in the dependency-chain sense, name the three structural sources in a React app, and explain why `Promise.all` is not a general fix.

A15 [18]. Split INP into its three phases and, for each, say whether the likely owner is React code, other main-thread work, or browser style/layout/paint.

A16 [19]. Define a memory leak precisely. Name four sources in React apps and explain why an `isMounted` flag does not fix a leaked subscription.

A17 [20]. What is a long task, why is `await Promise.resolve()` inside a loop not a yield, and what can React's concurrent rendering slice and not slice?

A18 [21]. Name the three Core Web Vitals, say what each approximates, and explain why field data (RUM) and lab data disagree. (State thresholds with the date you checked them.)

A19 [22]. List the steps of a hypothesis-driven production investigation, and explain Simpson's paradox in the context of an aggregate metric that "got worse".

A20 [23]. Apply Amdahl's law to a component that is 4% of an interaction's time, and give the five questions of the cost–benefit test.

Part B — Code and trace prediction (30 pts)

10 items, 3 points each. Predict *exactly*, then name the rule that explains it.

B1 [04, 05]. `labels` has 100 items. After mount, the button is clicked 3 times. How many times does `Row`'s function body run **in total including the initial mount**? Then: how many if `style` is replaced by a module-level constant?

```
const Row = memo(function Row({ label, style }: { label: string; style: React.CSSProperties })
{
  return <li style={style}>{label}</li>;
});

function List({ labels }: { labels: string[] }) {
  const [n, setN] = useState(0);
  return (
    <>
      <button onClick={() => setN(n + 1)}>{n}</button>
      <ul>
        {labels.map((l) => (
          <Row key={l} label={l} style={{ color: "red" }} />
        ))}
      </ul>
    </>
  );
}
```

B2 [06]. The button is clicked twice. How many times does `Label` render in total including mount? Is the answer different if `Label` is not wrapped in `memo`? What minimal change makes `Label` render once?

```tsx
const Ctx = createContext({ count: 0, label: "x" });

function Provider({ children }: { children: React.ReactNode }) {
  const [count, setCount] = useState(0);
  return (
    <Ctx.Provider value={{ count, label: "x" }}>
      <button onClick={() => setCount((c) => c + 1)}>+</button>
      {children}
    </Ctx.Provider>
  );
}

const Label = memo(function Label() {
  const { label } = useContext(Ctx);
  return <span>{label}</span>;
});

export function App() {
  return (
    <Provider>
      <Label />
    </Provider>
  );
}
```

B3 [13]. The user types one more character, changing `q` from `"a"` to `"ab"`, and nothing interrupts. How many times does `SearchPage` render, how many times does `Results` render, and with which `query` prop values (in order)? Assume `Results` is slow.

```tsx
const Results = memo(function Results({ query }: { query: string }) {
  /* slow */ return <ul>{filter(query).map((x) => <li key={x}>{x}</li>)}</ul>;
});

function SearchPage() {
  const [q, setQ] = useState("a");
  const deferred = useDeferredValue(q);
  return (
    <>
      <input value={q} onChange={(e) => setQ(e.target.value)} />
      <Results query={deferred} />
    </>
  );
}
```

B4 [07]. `Expensive` is not memoised. After mount and 3 button clicks, how many times has `Expensive` rendered in each variant?

```
// V1
function App() {
  const [c, setC] = useState(0);
  return (<><button onClick={() => setC(c + 1)}>{c}</button><Expensive /></>);
}
// V2
function Counter({ children }: { children: React.ReactNode }) {
  const [c, setC] = useState(0);
  return (<><button onClick={() => setC(c + 1)}>{c}</button>{children}</>);
}
function App2() { return <Counter><Expensive /></Counter>; }
// V3
function Counter3() {
  const [c, setC] = useState(0);
  return <button onClick={() => setC(c + 1)}>{c}</button>;
}
function App3() { return (<><Counter3 /><Expensive /></>); }
```

B5 [11, 20]. Write the exact console output order (all in one script run in a browser module):

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
queueMicrotask(() => console.log("D"));
(async () => { console.log("E"); await null; console.log("F"); })();
console.log("G");
```

B6 [15, 17]. Count the **serial network hops after** `main.js` **is running** until the first order rows with product names are on screen (assume `accountId` is in the URL, all queries fetch on mount, and `OrderRow`s mount together). Then say what you would change so the count becomes at most 2.

```tsx
const OrdersPage = lazy(() => import("./OrdersPage"));      // route chunk

// OrdersPage.tsx
export default function OrdersPage() {
  const { accountId } = useParams();
  const acct = useQuery(accountQ(accountId!));
  if (!acct.data) return <Spinner />;
  return <Orders accountId={acct.data.id} />;
}
function Orders({ accountId }: { accountId: string }) {
  const orders = useQuery(ordersQ(accountId));
  if (!orders.data) return <Spinner />;
  return <>{orders.data.map((o) => <OrderRow key={o.id} order={o} />)}</>;
}
function OrderRow({ order }: { order: Order }) {
  const product = useQuery(productQ(order.productId));
  return <li>{product.data?.name ?? "…"}</li>;
}
```

B7 [19]. The component mounts with `count = 0`, `count` changes to `1` and then `2`, then it unmounts. How many intervals are still running at the end? What does the interval keep reachable?

```tsx
function Ticker({ count }: { count: number }) {
  useEffect(() => {
    const id = setInterval(() => console.log("tick", count), 1000);
  }, [count]);
  return <span>{count}</span>;
}
```

B8 [18]. Assume the DOM is dirty (styles changed) before each snippet starts and `els` has 50 elements. How many **forced synchronous layouts** does each snippet cause?

```tsx
// (a)
for (const el of els) { const w = el.offsetWidth; el.style.width = `${w + 10}px`; }
// (b)
for (const el of els) { el.style.width = "200px"; }
const h = els[0].offsetHeight;
// (c)
let total = 0;
for (const el of els) { total += el.offsetHeight; }
```

B9 [21, 17, 18]. Field attribution for one page shows: LCP sub-parts TTFB 0.4 s, resource load delay 1.9 s, resource load duration 0.3 s, element render delay 0.5 s; and for the worst interactions INP = input delay 30 ms + processing 45 ms + presentation delay 280 ms. Which sub-part and which phase dominate, and which earlier lesson is your first stop for each?

B10 [04, 05, 23]. Parent re-renders because of **unrelated** state. For which of these (a)–(e) does `memo` actually skip the child's render? Give a one-line reason for each.

```
// (a)  <Row item={item} onSelect={() => select(item.id)} />                // Row =
memo(...)
// (b)  <Row item={item} onSelect={select} />                               // select =
useCallback(..., []), item from props, Row = memo(...)
// (c)  <Card><Heavy /></Card>                                              // Card =
memo(...)
// (d)  <Chart data={rows.filter((r) => r.active)} />                       // Chart =
memo(...)
// (e)  const active = useMemo(() => rows.filter((r) => r.active), [rows]);  <Chart data=
{active} />   // Chart = memo(...)
```

Part C — Debugging challenges (30 pts)

6 items, 5 points each. For each: state the root cause, name a concrete confirmation method (tool or experiment), and give the fix including at least one edge case.

C1 [01, 02, 18]. A data table (8,000 rows × 8 columns, `table-layout: auto`) feels slow when the user clicks "Apply filters". The React Profiler reports **30 ms** total for the commit (20 ms render, 10 ms commit effects). The Performance panel's Interactions track shows **input delay 5 ms, processing 45 ms, presentation delay 380 ms**. In the Main track after the click task you see Recalculate Style 60 ms ("22,000 elements affected"), Layout 240 ms, Paint 40 ms. A teammate proposes wrapping rows in `memo`. Diagnose.

C2 [17]. Waterfall for the checkout page on throttled mobile (ms from navigation start):

Request	Start–end
`GET /checkout/42` (HTML)	0–300
`main.js`	300–520
`checkout.chunk.js` (lazy route)	520–700
`GET /api/session`	700–850
`GET /api/carts/42`	850–1000
`GET /api/carts/42/shipping-options`	1000–1150
`hero.avif` (URL comes from the cart response)	1000–1400

Cart id `42` is in the URL; neither cart nor shipping-options needs the session response. Identify the chain(s), count hops after HTML, and propose changes that remove at least three hops.

C3 [19]. After opening and closing a "details drawer" 10 times, a heap-snapshot comparison (after forced GC, three-snapshot procedure) shows `+10 DrawerContent` closures and `+10 Detached HTMLDivElement`. Retainers for a detached div: `Map` → `drawerRegistry` (module scope in `registry.ts`) → `(internal array)` → `HTMLDivElement`. The relevant component:

```tsx
export function Drawer({ id }: { id: string }) {
  const ref = useCallback((el: HTMLDivElement | null) => {
    if (el) drawerRegistry.set(id, el);
  }, [id]);
  return <div ref={ref} className="drawer" />;
}
```

Find the leak, confirm it, fix it (including what happens when `id` changes while mounted), and say how you would prevent regression.

C4 [20]. In a chat sidebar page, key presses feel laggy (200–350 ms) but only while the sidebar is open. The React Profiler shows nothing on keypress. A Performance trace of typing shows the Interactions track with **input delay 180–320 ms** and tiny processing time. Every ~5 s there is a 250 ms task from a function `refreshNotifications` that parses a 3 MB JSON response. Diagnose and fix.

C5 [21, 22]. Field data, `/reports`, p75 INP before and after release 52:

Segment	Share before	Share after	p75 before	p75 after	n per week (each period)
Mobile	40%	60%	300 ms	305 ms	≥ 10,000
Desktop, standard accounts	55%	37%	120 ms	122 ms	≥ 10,000
Desktop, large accounts	5%	3%	150 ms	260 ms	≈ 1,500

The aggregate p75 INP moved from 190 ms to 235 ms and a manager asks you to "revert release 52". What do you conclude and what do you do?

C6 [03, 05, 13]. A product page "uses `useDeferredValue` for the filter, but typing is still janky and CPU stays busy after the user stops typing". Diagnose from the code:

```tsx
function Products({ products }: { products: Product[] }) {
  const [filters, setFilters] = useState({ q: "", category: "all" });
  const deferred = useDeferredValue({ ...filters });
  return (
    <>
      <input value={filters.q} onChange={(e) => setFilters((f) => ({ ...f, q: e.target.value
}))} />
      <ProductGrid products={products} filters={deferred} />
    </>
  );
}
function ProductGrid({ products, filters }: { products: Product[]; filters: Filters }) {
  const visible = products.filter((p) => matches(p, filters)); // slow with 20k products
  return <ul>{visible.map((p) => <ProductCard key={p.id} product={p} />)}</ul>;
}
```

Part D — Implementation exercises (30 pts)

5 items, 6 points each. Code must be TypeScript that would compile under `strict`. "Resource-safe" means timers, listeners, observers, and in-flight work are cleaned up.

D1 [14]. Implement `useDebouncedCallback<A extends unknown[]>(fn: (...args: A) => void, delayMs: number)` returning a stable debounced function with `.cancel()` and `.flush()` methods. Requirements: always calls the **latest** `fn`; trailing-edge only; cancels any pending call on unmount; `flush()` runs the pending call immediately (no-op if none); no re-creation of the returned function unless `delayMs` changes.

D2 [20]. Implement `processInChunks<T, R>(items: readonly T[], fn: (item: T, i: number) => R, opts?)` where `opts` has `budgetMs` (default 8), `signal?: AbortSignal`, and an injectable `yieldFn?: () => Promise<void>` (default: a macrotask-based yield). It must yield by **elapsed time**, reject with an `AbortError` `DOMException` when aborted, and be testable without real timers.

D3 [19]. Implement `useInterval(callback: () => void, delayMs: number | null)` which (a) always invokes the latest callback, (b) pauses when `delayMs` is `null`, (c) does not restart the timer when only `callback` identity changes, and (d) cleans up on unmount and on `delayMs` change.

D4 [17]. Rewrite this loader to remove the artificial waterfall. Requirements: `account` and `transactions` are required (failure of either fails the page and cancels the other request); `alerts` are optional (failure yields `null` and must not fail the page); everything must be cancellable through a caller-supplied `AbortSignal`.

```
async function loadAccountPage(accountId: string) {
  const account = await getAccount(accountId);
  const transactions = await getTransactions(accountId);
  const alerts = await getAlerts(accountId);
  return { account, transactions, alerts };
}
// Assume: getX(accountId: string, init?: { signal?: AbortSignal }): Promise<…>
```

D5 [21]. Implement `createRumReporter` for Web Vitals: `createRumReporter({ endpoint, sampleRate, release, getRoute })` returns `(metric: { name: string; value: number; id: string; rating: string }) => void`. Requirements: the sampling decision is made **once per session** (not per metric) and survives route changes (use `sessionStorage` but cope with it throwing); the payload includes `route` (from `getRoute()`), `release`, and `sampleRate` so the backend can weight counts; delivery uses `navigator.sendBeacon` with a `fetch(..., { keepalive: true })` fallback; no PII is added.

Part E — Architecture and engineering decisions (20 pts)

4 items, 5 points each. Answer in a structured paragraph or bullet list (150–300 words).

E1 [07, 08, 09, 10, 12]. An internal analytics tool must show up to 20,000 rows with text filtering, sorting by any column, multi-row selection, keyboard navigation, find-in-page expectations, and screen-reader

support. Typing in the filter must feel instant on a mid-range laptop. Choose an approach (client-side vs server-side processing, virtualisation vs pagination, transitions/deferral, workers) and defend it with criteria, trade-offs, and a measurement plan.

E2 [15, 16, 17, 21]. A consumer SPA has p75 mobile LCP 4.2 s ("poor" is above 4 s on web.dev), a 620 kB gzip entry bundle, and a client-rendered product page whose hero image URL arrives in an API response. Propose a prioritised plan with expected effects and risks. State which numbers you must measure first.

E3 [21, 22]. Design the performance operating model for a company with 10 product teams: field metrics, budgets (types, owners, enforcement), alerts, investigation process, and how you prevent budget gaming.

E4 [04, 23]. A team wants to ban manual `useMemo` / `useCallback` / `memo` ("the compiler will handle it") while another wants a lint rule requiring them everywhere ("to be safe"). Evaluate both positions, state what would change your mind, and propose a policy. Mention what must be verified against current React Compiler documentation.

---

6. Answer key

> Attempt every part before reading this section.

6.1 Part A — answers

A1. Unreliable because: (1) a development build is much slower and instrumented (extra checks, Strict Mode double rendering, source maps) so absolute numbers do not transfer; (2) a fast laptop with no throttling hides CPU-bound problems that appear on mid-range mobile; (3) a single run is noisy (JIT warm-up, GC, caches) and unrepresentative; (4) the Profiler measures React render time only, not layout/paint/network/third parties; also (5) extensions/DevTools open add overhead; (6) data size may be unrealistic. A defensible setup: production build (profiling alias if you need the React Profiler), incognito without extensions, CPU throttling 4–6× and a network profile, realistic data volume, a defined metric and baseline, multiple runs with medians, and field data to confirm. *Scoring (2):* 1 for ≥ 3 valid reasons; 1 for a defensible setup (at least production build + throttling + repeated runs).

A2. It measures, per commit, which components rendered and how long their render (and the commit's React-side work) took, plus why they rendered (when enabled). It cannot see: layout/style recalculation/paint/compositing; network time and waterfalls; other JS (third-party scripts, non-React handlers); GC pauses; input delay due to other long tasks; memory growth. Any three. *Scoring:* 1 core definition; 1 three valid blind spots.

A3. A component renders again when (1) its own state changes (`useState` / `useReducer`), (2) its parent renders (default: children re-render regardless of props unless `memo` /same-element bailout applies), or (3) a context it consumes changes value. "Props unchanged" is not a reason to skip because React by default re-renders children whenever the parent renders; only `memo` (or receiving the exact same element reference as `children` from a non-re-rendering ancestor) lets the child bail out. *Scoring:* 1 for the three triggers; 1 for parent-render default (and `memo` as the opt-out).

A4. `memo` skips a child's render when the component is wrapped, the parent re-rendered, **every prop is** `Object.is` **-equal** to the last render's props, and it did not itself receive a state/context update. It helps when the child subtree is expensive and parent renders are frequent with unchanged props. On every parent render where it fails to skip, you pay a props comparison (O(#props)) plus the render, a small but non-zero overhead; if props are unstable it never skips, and you have pure overhead. *Scoring:* 1 conditions; 1 cost on failure.

A5. New on every render: object literals, array literals (and results of `.map/.filter/.sort` copies), inline functions, JSX `children` /elements, `new Date()` / `new Map()` etc. Stable options: hoist constants to module scope; `useMemo` / `useCallback` with correct deps; pass primitives instead of objects; derive inside the child; restructure so the value lives in state or comes from a stable source; for `children`, pass elements from a non-re-rendering ancestor. Accept any three kinds with two options. *Scoring:* 1 three kinds; 1 two valid remedies (applied correctly).

A6. Context consumers re-render whenever the provider's `value` changes identity (`Object.is`), even if the field they read is unchanged — all 300 re-render every second. `memo` does not help because context updates bypass props comparison for the consuming component (the context dependency forces the render). Remedies: split the context by update frequency (or move the high-frequency value out); memoise/stabilise the value if the contents don't change; use an external store with selector subscriptions (`useSyncExternalStore`) so only consumers of changed slices re-render; colocate the fast-changing value in a leaf component; pass slow-changing data via props/children composition. *Scoring:* 1 mechanism (identity + all consumers; memo irrelevant); 1 two remedies.

A7. *Colocation:* keep state in the lowest component that needs it (or lift only as high as necessary) so state updates re-render a small subtree; the `children` variant moves the state into a wrapper that receives the rest of the tree as `children`, so the expensive elements keep the same identity when the wrapper re-renders and React bails out (same element reference). For 5,000 rows re-rendered per keystroke the cost is dominated by: (1) per-row render function work × 5,000 (and any hooks per row, e.g. queries/subscriptions); (2) reconciliation/commit work and DOM mutations; (3) browser style/layout/paint/memory cost of thousands of DOM nodes; (also key churn). Any three. *Scoring:* 1 colocation + children variant; 1 three costs.

A8. Windowing renders only the rows intersecting the viewport (plus overscan), uses a container with total scroll height (spacer/padding or absolutely positioned items) and recomputes the window on scroll/resize (using fixed or measured row heights). Poor default for 100 rows because: complexity and bugs (variable heights, scroll restoration, dynamic content), a11y/find-in-page/anchor limitations (rows not in the DOM), blank flashes on fast scroll, focus management, and negligible savings when the DOM is already cheap — fixed costs exceed the benefit. *Scoring:* 1 mechanism; 1 three costs.

A9. Reasonable order: (1) algorithm/data-structure change (e.g. build an index, avoid $O(n^2)$) — removes work, no new moving parts; (2) compute at update time or on the server rather than during every render — avoids repeated work; (3) `useMemo` — only caches when deps are stable and doesn't reduce the first computation's cost; (4) worker (or chunking) if still > ~50–100 ms and data is transferable — highest complexity, startup and serialisation cost. Accept a different order with sound justification (e.g. "move to update time" first) as long as "remove work before moving it" is respected. *Scoring:* 1 sensible order; 1 justification (remove before relocate; complexity/overhead).

A10. `useTransition` marks the state updates inside `startTransition` as non-urgent: React may render them in the background, interrupt them for urgent updates (typing), keep showing the previous UI while pending (`isPending`), and discard stale work. It does **not**: make a slow component faster, slice a single long component function, make the commit/DOM mutation interruptible, help non-React work, or reduce total work. The input's own state must remain urgent because a controlled input needs its value updated synchronously with the keystroke; deferring it causes dropped/laggy characters. *Scoring:* 1 guarantees and non-guarantees; 1 input urgency reason.

A11. On change, React first re-renders with the *old* deferred value (urgent pass: input updates, memoised slow child skipped because its prop is unchanged), then schedules a background render with the new value, which is interruptible and abandoned/restarted if the value changes again. No fixed delay. Slow consumer needs `memo` because otherwise the urgent pass re-renders it anyway (the parent rendered) with the old value, paying the cost at urgent priority and again in the background pass. *Scoring:* 1 two passes + interruptibility; 1 memo reason.

A12. Debounce waits for a quiet period then runs (trailing, optionally leading); throttle guarantees at most one run per interval. Use debounce for search-as-you-type network calls/autosave; throttle for scroll/resize/pointermove handlers. Debouncing a slow render adds latency to the result and, when it fires, the same render still blocks the main thread; it reduces frequency, not per-run cost or blocking — use algorithmic fixes, `useDeferredValue` /transitions, virtualisation. *Scoring:* 1 distinction + use cases; 1 why not for render.

A13. Good for large, rarely used, off-critical-path code (heavy routes, editors, charts, admin screens) behind `Suspense` with a good fallback. Backfires for tiny components, critical-path code, and nested lazy components: extra requests, serial hops/waterfalls, fallback flashes, and a discovered-late chunk. Bundle analysis: look at the treemap/size by module (parsed vs gzip), duplicated packages/versions, heavy dependencies, what's in the entry chunk vs lazy chunks. Tree-shaking defeaters: CommonJS modules, side-effectful module top-level code/missing `sideEffects` flag, barrel files re-exporting everything, dynamic `require` , namespace imports used dynamically (any two). *Scoring:* 1 when/backfire; 1 two analysis items + two defeaters (needs all four for 1).

A14. A waterfall is a chain of requests where each cannot start until the previous completes, so total time ≈ sum of hops (RTT + server + transfer). Sources: (1) resource discovery chains (CSS `@import` , JS-discovered imports/images/fonts), (2) render-then-fetch chains (child fetches only after parent's data renders it), (3) code-then-data (lazy chunk must load before its data fetch starts). `Promise.all` only parallelises requests whose inputs are already known; it cannot parallelise a request that needs another's response and does nothing for fetch-on-render chains. *Scoring:* 1 definition + three sources; 1 `Promise.all` limitation.

A15. Input delay (event → handler starts): other main-thread work (long tasks, third parties, hydration, GC). Processing time (handlers): your code and React's synchronous render/commit. Presentation delay (handler end → next paint): browser style/layout/paint/compositing (and any remaining work), large DOM, expensive CSS. *Scoring:* 1 three phases; 1 correct owners.

A16. A leak is memory that remains **reachable** from a GC root although the program no longer needs it, and which grows with repeated actions (not merely high memory or garbage awaiting collection). Sources: effects without cleanup (listeners, timers, observers, sockets), module-level caches/registries growing without bound, closures capturing large objects, detached DOM nodes held by refs/maps/libs,

unbounded state arrays, third-party instances without `destroy()`, object URLs not revoked. An `isMounted` flag only stops `setState`; the listener/timer remains registered and keeps its closure (and what it captured) reachable — you must unsubscribe/cancel. *Scoring:* 1 definition + four sources; 1 isMounted reasoning.

A17. A long task is a task occupying the main thread ≥ 50 ms (Long Tasks API). `await Promise.resolve()` continues in a microtask, which drains before the browser can process input or render, so the loop remains one task; yielding needs a macrotask (e.g. `setTimeout`, `MessageChannel`, `scheduler.yield()` where available: Chrome 129+, Firefox 142+, not Safari). React can slice *non-urgent render work between fibers* (transitions, deferred values, Suspense retries); it cannot slice a single component function, event handlers, effects, commit/DOM mutation, or non-React JS; urgent renders from discrete events run synchronously. *Scoring:* 1 long task + why await isn't a yield; 1 React slice/not slice.

A18. *Thresholds verified against web.dev and the* `web-vitals` *README on 2026-10-22.* LCP (loading: when the main content appears; good ≤ 2.5 s, poor > 4 s), INP (responsiveness across interactions; good ≤ 200 ms, poor > 500 ms), CLS (visual stability; good ≤ 0.1, poor > 0.25), assessed at p75. Lab data (Lighthouse/DevTools) is controlled and good for diagnosis; field data (RUM/CrUX) reflects real devices, networks, third parties, interactions, long sessions — they disagree because the lab uses a fixed profile, cold cache, no interaction, and a different population. *Scoring:* 1 three metrics + meaning; 1 lab/field reasons.

A19. Observe/report → scope (what, where, who, since when, how much) → build the change timeline → mitigate if severe and safe (rollback/flag-off) → hypotheses with predictions → cheapest discriminating test → reproduce under matching conditions → root-cause statement → fix → verify (field cohort comparison) → prevent (test/budget/alert). Simpson's paradox: an aggregate worsens (or improves) because the **mix of segments** shifted (e.g. more mobile users) even though each segment is flat or better; compare within segments or reweight. *Scoring:* 1 steps (≥ 6 in order, including scope + verify); 1 Simpson's.

A20. Amdahl: overall speed-up is limited by the fraction of time affected; at 4% the maximum improvement is ≈ 4% (speedup ≤ 1/(1−0.04) ≈ 1.04×), and overhead added elsewhere can make it net negative. Five questions: (1) What's the measured bottleneck evidence? (2) What is the mechanism by which the change removes the cost? (3) What are the overheads/side effects? (4) What's the expected benefit vs noise and perceptual threshold? (5) How will we verify and roll back? *Scoring:* 1 Amdahl application; 1 five questions (≥ 4 correct).

6.2 Part B — answers

B1. `Row` renders **400 times**: 100 on mount + 3 clicks × 100 (`style={{ color: "red" }}` is a new object each render, so `memo`'s shallow comparison fails for every row). With a module-level `const RED = { color: "red" }` and `style={RED}`: **100 times** (clicks re-render `List`, but every `Row`'s props are `Object.is`-equal, so each is skipped). *Scoring:* 2 for both counts (1 if one is right); 1 for naming the rule (new object identity defeats shallow prop comparison).

B2. 3 renders (mount + 2). `Provider` re-renders on each click and creates a new `value` object `{ count, label }`; `Label` consumes the context, so it re-renders when the value's identity changes, even

though it reads only `label`. `memo` doesn't matter: without `memo` it's also 3 — the `children` element is the same reference (created in `App`, which doesn't re-render), so the only thing causing the re-render is the context change. Minimal change: put `count` in a separate context (or keep `label` in its own context/constant), or avoid putting changing data in the same value: e.g. `<LabelCtx.Provider value="x">` separately from `<CountCtx.Provider value={count}>`; or use a selector-based store. (Memoising the value with `useMemo(() => ({count, label}), [count])` does *not* help — `count` changes.) *Scoring:* 2 for 3 and the correct no-`memo` answer (1 if only the count); 1 for the context-identity rule and a valid fix.

B3. `SearchPage` renders **2 times** (urgent pass with `q = "ab"` and `deferred = "a"`; then the background pass where `deferred = "ab"`). `Results` renders **1 time**, with `query = "ab"` (in the urgent pass its prop `"a"` is unchanged, so `memo` skips it). Wrong answer: 2 renders for `Results` (that is what happens without `memo`: `"a"` then `"ab"`). *Scoring:* 2 for exact counts and the prop value; 1 for naming the two-pass/memo-skip rule.

B4. V1: **4** (mount + 3 re-renders because `App` re-renders and `Expensive` is not memoised). V2: **1** (the `<Expensive />` element is created in `App2`, which never re-renders; `Counter` re-renders but receives the same `children` element reference, so React bails out of `Expensive`). V3: **1** (state is colocated in `Counter3`; `App3` does not re-render). *Scoring:* 2 for all three counts (1 for two correct); 1 for naming colocation/children-identity.

B5. `A E G C D F B`. Synchronous logs first: `A`, then the async function's synchronous part logs `E` (it then awaits `null`, scheduling its continuation as a microtask), then `G`. Microtasks run in FIFO order of enqueueing: the `.then` callback (`C`), the `queueMicrotask` callback (`D`), and the async continuation (`F`). The `setTimeout` callback (`B`) is a macrotask and runs after microtasks drain. *Scoring:* 2 for exact order (1 if one adjacent pair is swapped); 1 for the rule (sync → microtasks FIFO → macrotask).

B6. After `main.js` is running: (1) route chunk (lazy), (2) `/accounts/:id` query, (3) `/accounts/:id/orders`, (4) the product queries (parallel siblings) = **4 serial hops**. Fix (≤ 2): start data at route match in parallel with the chunk (route loader/prefetch; both queries need only `accountId` from the URL) → hop 1 = chunk ‖ account ‖ orders; return product data inline with the orders (`?include=product`) or via a batch endpoint → no hop 2 for products; or at worst one more hop for products. Alternatively preload the chunk on link intent so the chunk hop leaves the critical path. *Scoring:* 2 for 4 hops and the correct fix class; 1 for reasoning about artificial dependency (`accountId` already known) and parallelism.

B7. 3 intervals are running at the end (mounted with count 0, then re-runs for 1 and 2, no cleanup returned so each effect run leaves its interval running), and **still 3 after unmount**. Each interval callback keeps its closure (`count` at the time) reachable and keeps logging forever. Fix: `return () => clearInterval(id)`. *Scoring:* 2 for 3 (and still 3 after unmount); 1 for reachability/cleanup reasoning.

B8. (a) **50** (every iteration reads after the previous write dirtied layout). (b) **1** (writes only, then a single read forces one layout). (c) **1** (the first read forces layout; subsequent reads with no intervening writes find layout clean). The problem is *interleaving*, not the count of reads. *Scoring:* 2 for all three (1 for two); 1 for the interleaving rule.

B9. LCP: **resource load delay (1.9 s)** dominates → 17 — Network waterfalls (late discovery/priority: put the LCP image in the HTML/preload it, `fetchpriority`, remove lazy loading, remove data→image chain). INP: **presentation delay (280 ms)** dominates → 18 — React vs browser bottlenecks (style/layout/paint

cost: DOM size, CSS, forced reflow), not React processing (45 ms) or input delay. *Scoring:* 2 for both dominant parts; 1 for the correct lesson/lever for each (both required).

B10. (a) **No** — inline arrow is a new function each render. (b) **Yes** — `select` is stable (`useCallback` with `[]`), `item` comes from props with stable identity. (c) **No** — `<Heavy />` is a new element each parent render, so `children` differs (unless `Card`'s parent didn't re-render). (d) **No** — `.filter` returns a new array every render. (e) **Yes** — `useMemo` returns the same array while `rows` is unchanged. *Scoring:* 2 for exactly {b, e} with correct reasoning on at least four; 1 for the unifying rule (shallow identity of every prop).

6.3 Part C — answers

C1. *Root cause (2):* The cost is **browser-side** (Recalculate Style 60 ms, Layout 240 ms, Paint 40 ms ≈ presentation delay 380 ms), caused by a very large DOM (8,000 × 8 ≈ 64,000+ cells; 22,000 elements affected) with `table-layout: auto` forcing content-dependent column layout across all rows. React work is only 30 ms; `memo` would reduce at most part of that 30 ms and not touch the 340 ms of style/layout/paint. *Confirmation (1):* Performance panel recording on a production build with CPU throttling: confirm that the interaction's presentation delay dominates and that Layout time scales with row count (compare with 800 rows); check `document.querySelectorAll("*").length`; test `table-layout: fixed` or `content-visibility: auto` in DevTools. *Fix (2):* reduce DOM nodes — virtualise rows (09) or paginate/server-filter; use `table-layout: fixed` with explicit column widths; consider `content-visibility: auto` with `contain-intrinsic-size`; keep a11y (ARIA grid roles, row counts) and find-in-page trade-offs in mind. Edge cases: variable row heights, selection/focus retention across virtualised rows, keyboard navigation. *Scoring:* 2 root cause (browser layout/DOM size, not React); 1 confirmation; 2 fix with a named edge case (1 without).

C2. *Chains:* HTML → `main.js` → lazy `checkout.chunk.js` (code-then-data) → `/api/session` → `/api/carts/42` → `/api/carts/42/shipping-options` and, separately, cart response → hero image (late discovery). After HTML: `main.js` (1), chunk (2), session (3), cart (4), shipping (5), hero image (5, starts when cart arrives) → about **5 serial hops** (image shares the last depth). *Fixes:* (1) start `/api/carts/42` and `/api/carts/42/shipping-options` at route match, in parallel with the chunk load, since cart id is in the URL (route loader or router-level prefetch); (2) drop the session dependency from the critical path (nothing needs it; fetch concurrently or read from cookie-backed server render); (3) make the hero image discoverable early: include the URL in HTML (SSR/SSG) or `<link rel="preload" as="image" fetchpriority="high">` if static, or return the image URL with initial data and preload it, and don't lazy-load it; (4) optionally `modulepreload` the checkout chunk or preload on intent/hover from the cart page. Result: hops = main.js (1) → [chunk ‖ session ‖ cart ‖ shipping ‖ hero] (2). Edge cases: shipping options might truly depend on cart contents — then aggregate server-side or accept 3 hops; avoid preloading wrong resources; auth redirects. *Scoring:* 2 chain identification and hop count (allow counting variants if stated); 1 confirmation (waterfall/Initiator column/Resource Timing); 2 fixes (≥ 3 hops removed) with an edge case (1 otherwise).

C3. *Root cause (2):* The callback ref registers the element in the **module-level** `drawerRegistry` but never deletes it. React calls the ref callback with `null` on unmount (and when the callback identity changes), and the code ignores that branch, so the `Map` retains the detached `<div>` (and its subtree and whatever it references) forever; cycles accumulate `+10 detached divs`. *Confirmation (1):* three-snapshot technique

with forced GC; follow the retainer path to `drawerRegistry`; Performance monitor "DOM Nodes" count rising per cycle; after the fix the delta is 0. *Fix (2):*

```
const ref = useCallback((el: HTMLDivElement | null) => {
  if (el) drawerRegistry.set(id, el);
  else drawerRegistry.delete(id);
}, [id]);
```

Edge case: when `id` changes while mounted, React calls the *old* callback with `null` and the *new* one with the element, so the old key is deleted and the new key set (correct because the callback closes over the right `id`); also guard against a later registration being deleted by an earlier unmount of a different instance with the same id (`if (drawerRegistry.get(id) === prevEl) delete`) — or avoid the global registry (use a context/store scoped to a provider). *Prevention:* a test counting registry size after mount/unmount (also under Strict Mode), a lint/review rule against module-level Maps holding DOM nodes, soak test checking DOM node counters. *Scoring:* 2 root cause; 1 confirmation; 2 fix + edge case (1 without).

C4. *Root cause (2):* Input delay is large while processing is tiny, so the main thread is busy with **other work** before the key handler runs: the notification poller's 250 ms task (parsing a 3 MB JSON on the main thread every 5 s) occupies the thread; keypresses arriving during it wait. React isn't involved. *Confirmation (1):* Performance trace while typing: find the long task preceding input, Bottom-up shows `JSON.parse` / `refreshNotifications`; disable the sidebar or the interval and verify input delay disappears; Long Tasks/LoAF attribution in the field. *Fix (2):* shrink the payload (pagination, deltas/`since` cursor, fields projection), parse in a Web Worker or via streaming, pause polling when the tab is hidden, switch to push (SSE/WebSocket) for incremental updates, and ensure the interval is cleaned up on unmount; if still heavy, chunk processing with yields. Edge cases: worker transfer cost (use `ArrayBuffer` / `response.arrayBuffer()` inside the worker), cancellation on unmount, ordering of out-of-order responses. *Scoring:* 2 root cause; 1 confirmation; 2 fix with edge case (1 without).

C5. *Conclusion (2):* There is **no site-wide regression**; the aggregate rose because of a **mix shift** (mobile share 40% → 60%, with mobile consistently slower): Simpson's-paradox-style aggregate movement — within mobile (300 → 305) and desktop-standard (120 → 122) the changes are within noise. **However**, desktop large accounts regressed 150 → 260 ms (≈ +110 ms) with n ≈ 1,500 — a real, segment-specific regression hidden by the aggregate. *Confirmation (1):* compare within segments, check confidence intervals/bootstraps, same-period release cohorts (canary vs control) for large accounts, and examine attribution (input delay/processing/presentation) for that segment. *Action (2):* Don't revert on the aggregate; open an investigation for large accounts (hypotheses: data-size-dependent work introduced in release 52 — an $O(n^2)$ computation, unbounded list render, more rows/columns by default; compare account-size buckets; reproduce with a large fixture); consider a targeted flag-off if impactful; also note the new traffic mix means the mobile experience is now the dominant UX and deserves its own budget. Edge cases: small n for large accounts (verify confidence), bot/segment definition changes. *Scoring:* 2 correct conclusion (mix shift + hidden large-account regression; 1 if only one); 1 confirmation; 2 action (specific, segment-aware).

C6. *Root causes (2):* (1) `useDeferredValue({ ...filters })` passes a **new object every render**, so the deferred value is never `Object.is`-equal to the previous one; React keeps scheduling a background re-render with a "new" value, causing perpetual extra work/CPU after typing stops. (2) `ProductGrid` is **not**

memoised, so it re-renders in the urgent pass (parent rendered) with the old deferred object, paying the 20k-item filter at urgent priority — typing stays janky — and again in the background pass. (3) Also the heavy `products.filter` runs on every `ProductGrid` render without memoisation. *Confirmation (1):* Profiler "Why did this render?" on `ProductGrid` in urgent commits; count commits after the user stops typing (should be zero); check that the deferred value identity changes each time (log `Object.is` in a test). *Fix (2):*

```
const deferredFilters = useDeferredValue(filters);          // defer the stable state object itself
const ProductGrid = memo(function ProductGrid({ products, filters }: Props) {
  const visible = useMemo(() => products.filter((p) => matches(p, filters)), [products, filters]);
  return <ul>{visible.map((p) => <ProductCard key={p.id} product={p} />)}</ul>;
});
```

Edge cases: `products` identity must be stable (else `memo` fails); consider virtualising if the commit of thousands of cards is itself long ([09]); show a stale indicator (`filters !== deferredFilters`). *Scoring:* 2 root causes (both object identity and missing `memo` ; 1 for one); 1 confirmation; 2 fix with edge case (1 without).

6.4 Part D — answers

D1. Reference solution

```tsx
import { useCallback, useEffect, useMemo, useRef } from "react";

type Debounced<A extends unknown[]> = ((...args: A) => void) & { cancel: () => void; flush: () => void };

export function useDebouncedCallback<A extends unknown[]>(
  fn: (...args: A) => void,
  delayMs: number,
): Debounced<A> {
  const fnRef = useRef(fn);
  useEffect(() => { fnRef.current = fn; });                    // latest fn without changing returned identity

  const timer = useRef<ReturnType<typeof setTimeout> | undefined>(undefined);
  const pendingArgs = useRef<A | null>(null);

  const cancel = useCallback(() => {
    if (timer.current !== undefined) clearTimeout(timer.current);
    timer.current = undefined;
    pendingArgs.current = null;
  }, []);

  const flush = useCallback(() => {
    if (timer.current === undefined || pendingArgs.current === null) return;
    clearTimeout(timer.current);
    timer.current = undefined;
    const args = pendingArgs.current;
    pendingArgs.current = null;
    fnRef.current(...args);
  }, []);

  const debounced = useCallback((...args: A) => {
    pendingArgs.current = args;
    if (timer.current !== undefined) clearTimeout(timer.current);
    timer.current = setTimeout(() => {
      timer.current = undefined;
      const a = pendingArgs.current;
      pendingArgs.current = null;
      if (a !== null) fnRef.current(...a);
    }, delayMs);
  }, [delayMs]);

  useEffect(() => cancel, [cancel]);                           // cancel on unmount

  return useMemo(() => Object.assign(debounced, { cancel, flush }), [debounced, cancel,
```

```
flush]);
}
```

Key criteria: latest-fn ref (no stale closure); stable identity per `delayMs`; clear timer on unmount; `flush` semantic; `cancel` clears pending args; types preserve argument tuple `A`. Acceptable variations: a layout effect for the ref assignment; a note that changing `delayMs` mid-pending does not reschedule the in-flight timer (or handle it). Common errors: recreating the debounced function every render (breaks debouncing because timer state is lost); using state for the timer; forgetting unmount cleanup; `fn` captured stale. *Scoring (6):* 3 main behavior (trailing debounce + latest fn); 1 edge cases (`flush` / `cancel` semantics, no-op when nothing pending); 1 types/API (`A` tuple, stable return); 1 cleanup.

D2. Reference solution

```ts
type ChunkOptions = {
  budgetMs?: number;
  signal?: AbortSignal;
  yieldFn?: () => Promise<void>;
  now?: () => number;                       // injectable clock for deterministic tests
};

const defaultYield = (): Promise<void> =>
  new Promise<void>((resolve) => {
    const { port1, port2 } = new MessageChannel();
    port1.onmessage = () => { port1.close(); resolve(); };
    port2.postMessage(undefined);
  });

export async function processInChunks<T, R>(
  items: readonly T[],
  fn: (item: T, index: number) => R,
  { budgetMs = 8, signal, yieldFn = defaultYield, now = () => performance.now() }:
ChunkOptions = {},
): Promise<R[]> {
  const out: R[] = new Array<R>(items.length);
  let sliceStart = now();
  for (let i = 0; i < items.length; i++) {
    if (signal?.aborted) throw new DOMException("Aborted", "AbortError");
    out[i] = fn(items[i], i);
    if (now() - sliceStart >= budgetMs) {
      await yieldFn();
      if (signal?.aborted) throw new DOMException("Aborted", "AbortError");
      sliceStart = now();
    }
  }
  return out;
}
```

Notes: a manual `DOMException("Aborted", "AbortError")` is portable across runtimes (`signal.throwIfAborted()` is an alternative where available). The injectable `yieldFn` and `now` allow tests with a fake clock (e.g. clock advances inside `fn`, assert that `yieldFn` was called N times) without real timers. Time-based (not count-based) slicing adapts to device speed. Edge cases: empty array; already-aborted signal; very cheap `fn` (checking the clock each iteration costs — acceptable, or check every k items); exceptions thrown by `fn` propagate. *Scoring (6):* 3 main (chunk + yield by elapsed time, returns results in order); 1 abort handling (before work and after yield, `AbortError`); 1 API/types (injectable yield, generics, no `any`); 1 testability/resource-safety (no leaked ports, no real timers needed).

D3. Reference solution

```
import { useEffect, useRef } from "react";

export function useInterval(callback: () => void, delayMs: number | null): void {
  const saved = useRef(callback);
  useEffect(() => { saved.current = callback; });     // latest callback; does not restart the
timer

  useEffect(() => {
    if (delayMs === null) return;                     // paused
    const id = setInterval(() => saved.current(), delayMs);
    return () => clearInterval(id);                   // unmount and delay change
  }, [delayMs]);
}
```

Criteria: dependency array is only `[delayMs]`; callback read through ref; `null` pauses; cleanup. Common errors: depending on `callback` (restarts the interval whenever identity changes, so a re-rendering parent never lets the interval fire); capturing the first callback only (stale closure); not returning cleanup. Note: assigning `saved.current` inside an effect (rather than during render) keeps render pure under concurrent rendering. (`useEffectEvent` is a designed solution for this pattern and it is stable as of React 19.2.) *Scoring (6):* 3 main behavior (a–c); 1 edge case (null pause/resume, delay change); 1 types; 1 cleanup.

D4. Reference solution

```ts
export async function loadAccountPage(accountId: string, callerSignal: AbortSignal) {
  const ac = new AbortController();
  const onAbort = () => ac.abort(callerSignal.reason);
  if (callerSignal.aborted) ac.abort(callerSignal.reason);
  else callerSignal.addEventListener("abort", onAbort, { once: true });

  const init = { signal: ac.signal };
  try {
    // start everything immediately
    const alertsP = getAlerts(accountId, init).catch((): null => null);     // optional: never rejects
    const [account, transactions] = await Promise.all([
      getAccount(accountId, init),
      getTransactions(accountId, init),
    ]);
    const alerts = await alertsP;
    return { account, transactions, alerts };
  } catch (e) {
    ac.abort();                       // cancel siblings still in flight (e.g. alerts, other required call)
    throw e;
  } finally {
    callerSignal.removeEventListener("abort", onAbort);
  }
}
```

Criteria: all three requests start in the same tick; required ones in `Promise.all` (rejects on first failure); an internal controller aborts the sibling requests when one required request fails (plain `Promise.all` does not cancel others); optional alerts caught so failure doesn't fail the page; caller cancellation propagated; listener cleanup. Edge cases: alerts failing due to abort should not produce unhandled rejections (the `.catch` handles it); `getAlerts` may reject with `AbortError` after cancellation — fine; if the caller wants alerts to arrive later without gating the page, return `alertsP` (a promise) and render it in its own Suspense boundary instead of awaiting it. Common errors: `Promise.allSettled` for everything (swallows required failures), sequential awaits, not cancelling siblings, leaking the abort listener. *Scoring (6):* 3 parallel + required/optional semantics; 1 edge cases (sibling cancellation, no unhandled rejection); 1 types (typed returns, `catch(): null`); 1 cleanup (listener removal, abort).

D5. Reference solution

```ts
type RumMetric = { name: string; value: number; id: string; rating: string };

type RumOptions = {
  endpoint: string;
  sampleRate: number;                    // 0..1
  release: string;
  getRoute: () => string;                // route TEMPLATE, e.g. "/orders/:id"
};

const SAMPLED_KEY = "rum:sampled";

function decideSampled(sampleRate: number): boolean {
  try {
    const existing = sessionStorage.getItem(SAMPLED_KEY);
    if (existing !== null) return existing === "1";
    const decision = Math.random() < sampleRate;
    sessionStorage.setItem(SAMPLED_KEY, decision ? "1" : "0");
    return decision;
  } catch {
    return Math.random() < sampleRate;   // storage unavailable: per-page-load decision (see note)
  }
}

export function createRumReporter({ endpoint, sampleRate, release, getRoute }: RumOptions) {
  let sampled: boolean | undefined;       // memoised per page load; sessionStorage makes it per session

  return function report(metric: RumMetric): void {
    sampled ??= decideSampled(sampleRate);
    if (!sampled) return;

    const body = JSON.stringify({
      name: metric.name,
      value: metric.value,
      rating: metric.rating,
      id: metric.id,
      route: getRoute(),
      release,
      sampleRate,                         // backend weights counts by 1 / sampleRate
    });

    const sent = typeof navigator.sendBeacon === "function"
      && navigator.sendBeacon(endpoint, new Blob([body], { type: "application/json" }));
    if (!sent) {
      void fetch(endpoint, {
```

```
        method: "POST",
        body,
        keepalive: true,
        headers: { "content-type": "application/json" },
      }).catch(() => { /* telemetry must never throw into the app */ });
    }
  };
}
```

Criteria: decision made once and persisted per session, with `try/catch` around storage; `sampleRate` in payload; route template via callback, not `location.pathname`; `sendBeacon` with a fallback to `fetch` with `keepalive`; swallow telemetry errors; no PII (no URLs with IDs, no selectors containing user data). Note: when storage is unavailable the fallback decision is made once per page load (memoised in the closure), which is acceptable but biases multi-page sessions; document it. *Scoring (6):* 3 main (sampling + payload + delivery); 1 edge cases (storage throws, beacon returns false); 1 types; 1 safety (errors swallowed, no PII, no per-metric re-sampling).

6.5 Part E — answers (evaluation criteria and example responses)

E1. Criteria. (1 pt) Alternatives considered: client-side everything with virtualisation; server-side filtering/sorting/pagination; hybrid (client-side with a worker for filtering); `content-visibility`; transitions/deferral. (2 pts) Causal reasoning against requirements: 20k rows on mid-range laptops — filtering 20k strings is cheap if algorithmically sound (precomputed lowercase fields or an index; typically a few ms to tens of ms), but *rendering* 20k rows is expensive in DOM/style/layout/memory (18); memoisation does not fix DOM size; virtualisation reduces DOM but breaks native find-in-page and needs ARIA grid semantics (`aria-rowcount`, row indices), focus retention and keyboard navigation work; pagination preserves native behavior but fragments selection and sorting unless server-side; transitions/deferral keep typing responsive but cannot slice the commit of many nodes; a worker only helps if compute (not render) dominates. (1 pt) Clear decision. (1 pt) Measurement and risk plan. *Example answer:* Primary: client-side data (if the dataset fits memory comfortably) with a precomputed search index and `useDeferredValue` for the results, **virtualised rows** with fixed or estimated heights and ARIA grid roles; keep input state urgent and the memoised row component stable (13, 09). Provide an in-app "search all rows" box because browser find-in-page can't see unrendered rows (documented trade-off for an internal tool). If users need cross-page selection and exact find semantics or data exceeds ~100k rows/exceeds memory budget, move filtering/sorting server-side with cursor pagination. Rejected: render 20k rows + `memo` (DOM cost remains); debounce (adds latency without reducing cost); worker for filtering unless profile shows > 50 ms. Measurement: production build, CPU 4× throttle, 20k-row fixture, record INP-style latency for typing (target e.g. ≤ 100 ms p95 in lab), DOM node count, memory; screen-reader smoke test with NVDA/VoiceOver; field INP by route afterwards. Risks: variable row heights, accessibility regressions, selection model complexity.

E2. Criteria. (1) Alternatives: SSR/SSG for the product page; preload/inline hero; bundle reduction; route/vendor splitting; CDN/caching; image optimisation. (2) Reasoning: use the LCP sub-parts first — likely large *resource load delay* (hero URL arrives via API: client-render → JS → API → image chain, 17) plus *render delay* from a 620 kB gzip entry; measure TTFB, the four sub-parts, parse/execute cost on a mid-range device, and what's in the bundle (16). Plan by expected effect per effort: (a) make the hero

discoverable early (server-render/inline the product data and image URL or preload; `fetchpriority="high"`; no lazy-loading on it; correct dimensions) — directly attacks load delay; (b) reduce entry JS: analyse the treemap, lazy-load heavy rare features, replace/trim large dependencies, avoid barrel imports, `modulepreload` the route chunk; (c) parallelise the data (route loader; aggregate endpoint) so product data isn't behind chunk → effect; (d) image format/size/CDN caching; (e) only then consider deeper changes (SSR framework, RSC) with a cost/risk discussion (Phase 9). (1) Decision/prioritisation. (1) Plan to measure: lab with throttled profile and field RUM p75 LCP before/after by route/device; budgets (size-limit) to keep the gains; risks: SSR hydration costs/mismatches, cache invalidation, CMS image constraints. State thresholds with "verify on web.dev". Accept any well-justified prioritisation that attacks the biggest measured sub-part first.

E3. Criteria. (1) Alternatives: central perf team vs federated ownership; hard vs soft budgets; vendor RUM vs self-built. (2) Reasoning: field metrics (LCP/INP/CLS at p75 by route template, device class, release; custom task metrics for key journeys) as the SLO layer; lab checks and deterministic budgets (bundle size per entry, request counts, third-party time) as PR gates; soft lab timing checks relative to main; ownership per route/team with a platform-owned shell budget; alerting with minimum sample sizes and duration; investigation runbook (22): scope → change timeline → hypotheses → verification by cohort; budget ratchet and expiring exceptions via ADRs; third-party governance. Anti-gaming: pair vitals with business KPIs and visual checks; review LCP element attribution; avoid single-score targets; monitor segment tails (p95) and custom metrics; audit exceptions. (1) Clear model. (1) Measurement/risk plan (alert fatigue, flaky CI, sampling bias, privacy: route templates not URLs, no PII in selectors). *Example:* start small: web-vitals RUM → route/release dashboard; three hard CI gates (entry size, per-route chunk size, third-party count); two soft gates; on-call rotation; quarterly budget review.

E4. Criteria. (1) Alternatives: ban, require, evidence-based default, compiler adoption with escape hatches. (2) Reasoning: *Ban* is brittle: manual memoisation remains useful for effect-dependency stability, library components, intentionally expensive calculations, and projects where the compiler isn't enabled or code violates the Rules of React (so compilation bails out); *require everywhere* adds noise, stale-dependency bugs, overhead, and often no benefit (23); structural fixes (colocation, composition) beat both. Policy: default to no manual memoisation in application code; add `memo` / `useMemo` / `useCallback` only with evidence (profile link) or for documented reasons (effect deps; public library components; measured expensive computation); keep `react-hooks/exhaustive-deps` enforced; if adopting the compiler, adopt incrementally, run its lint/diagnostic tooling, keep tests/benchmarks, and remove manual memoisation only where parity is measured. (1) Decision; (1) what would change my mind (benchmarks showing wins from manual memo in hot paths; compiler bailouts in large parts of the codebase; version constraints) and a measurement plan (render-count tests, Profiler comparisons, field INP). **Must be verified against current docs:** supported React versions, how to enable/opt out per file or component, which patterns cause bail-outs, what lint rules exist, and how it interacts with libraries (https://react.dev/learn/react-compiler; the compiler reached stable v1.0 in October 2025).

---

7. Scoring rubric

7.1 Points and categories

Scores are tallied into **four categories**. A passing result requires **≥ 80% overall (120/150)** *and* **≥ 70% in every category**.

Category	What it measures	Parts	Max points	70% floor
Explanation	Can you state and justify the concepts precisely?	Part A	40	28
Reasoning	Can you predict behavior and make justified trade-off decisions?	Part B (30) + Part E (20)	50	35
Debugging	Can you find root causes, confirm them with evidence, and verify fixes?	Part C	30	21
Implementation	Can you write correct, typed, resource-safe code?	Part D	30	21
Total			**150**	**120 overall**

7.2 Per-item scoring rules

Part	Points per item	Breakdown
A	2	1 = core fact(s) correct. 1 = mechanism/consequence/example correct. Right words with a wrong mechanism: at most 1.
B	3	2 = exact prediction (1 if partially right, 0 if the approach is wrong). 1 = the correct rule named (shallow identity, context identity, two-pass, microtask ordering, interleaved read/write...).
C	5	2 = root cause (symptom restatement alone earns at most 1). 1 = a concrete confirmation method (tool/experiment, not "debug it"). 2 = correct fix including a named edge case (1 for a fix that works but ignores edge cases).
D	6	3 = main behavior correct. 1 = edge cases handled (named in the key). 1 = types/API design sound (no unnecessary `any` /casts). 1 = cleanup/resource safety (timers, signals, listeners, no unhandled rejections). Code that doesn't compile/run earns at most 3.
E	5	1 = alternatives considered. 2 = criteria and causal reasoning (not assertion). 1 = clear decision. 1 = measurement/risk plan.

Self-scoring discipline. Credit only what you wrote *before* checking the key. Where the key lists multiple acceptable answers, any of them counts. Thresholds for Web Vitals count as correct if they match the current web.dev values even if they differ from this key (thresholds can change). When unsure, take the lower score.

7.3 Result bands

Overall	Category floors met?	Result
≥ 135 (90%)	Yes	**Distinction:** proceed; you can lead performance investigations and review optimisation PRs critically.

Overall	Category floors met?	Result
120–134	Yes	**Pass:** proceed. Revisit lessons tied to your lowest items.
≥ 120	**No** (any category < 70%)	**Conditional:** remediate the failing category (§8) and retake only that category's parts with fresh variants.
100–119	—	**Not yet:** targeted remediation, then full retake after at least 7 days.
< 100	—	**Return to lessons:** redo the exercises (not only reading) for the mapped lessons, then full retake.

8. Promotion criteria and remediation

8.1 Promotion criteria

You are ready to move on from Phase 7 when **all** of the following hold:

1. **Score gate:** ≥ 120/150 overall and ≥ 70% in each category (§7).
2. **Critical-concept gate:** at least 80% of the available points within each of these clusters (they are used constantly in Phases 8–11):
   - *Measure before changing* (A1, A2, A18, A19, A20, C1, C5),
   - *Why and when React re-renders / what skipping requires* (A3, A4, A5, A6, A7, B1, B2, B4, B10, C6),
   - *Main thread, scheduling, and async ordering* (A10, A11, A17, B3, B5, C4, D2),
   - *Network and memory lifetime* (A14, A16, B6, B7, C2, C3, D3, D4).
3. **Practice gate:** without references you can (a) record and read a Performance-panel trace and classify an interaction by input delay / processing / presentation delay, (b) take a three-snapshot heap comparison and follow a retainer path, (c) read a waterfall and name the dependency chain, and (d) write a PR justification for an optimisation using the five-question cost–benefit test.
4. **Hands-on gate:** you have completed (or are mid-way through) the Phase 7 projects (Project 5 — Data-heavy interface; Project 11 — Broken app) with at least one regression you detected from a measurement rather than by reading code.

If you pass the score gate but miss a critical cluster, proceed only after a focused 30–45 minute review of the lessons below, because those concepts underpin testing strategy (budgets and render-count tests), SSR/hydration work, and incident debugging.

8.2 Remediation map

If you lost points on...	Re-study	Then do
A1, A2, A20, C1	01, 02, 23	Profile a small app in dev vs production with and without CPU throttling; write a one-page before/after report

If you lost points on...	Re-study	Then do
A3, A4, A5, B1, B10, C6	03, 04, 05	Instrument 5 components with the Profiler; predict which re-render before running; fix unstable props
A6, B2	06	Build a context with a 1 Hz field and 200 consumers; fix it three ways (split, selector store, colocation) and compare commits
A7, B4	07, 08	Refactor a root-level state tree using colocation and `children` composition; count renders before/after
A8, E1	09, 10	Render 20k rows plain vs windowed; measure DOM nodes, INP-style latency, and accessibility gaps
A9, A10, A11, B3	10, 11, 12, 13	Build a slow filtered list; compare plain / transition / deferred+memo; record commits per keystroke
A12, D1	14	Implement debounce/throttle hooks with unmount cleanup; test with fake timers
A13, E2	15, 16	Analyse a real bundle; split one heavy route; show the waterfall before and after
A14, B6, C2, D4, E2	17	Flatten a deliberately nested-fetch page; show hop counts before/after in the Network panel
A15, B8, B9, C1, C4	18, 20	Trace a layout-thrash loop and its fix; classify ten interactions by phase
A16, B7, C3, D3	19	Reproduce a listener leak and a detached-node leak; fix and verify with three-snapshot deltas
A17, C4, D2	20	Chunk a 400 ms computation with time-based yielding; move another to a worker; compare input delay
A18, B9, C5, D5, E3	21	Add `web-vitals` with attribution to a sample app; define two budgets and a CI gate; verify thresholds on web.dev
A19, C5, E3	22	Write an investigation log (scope, timeline, hypotheses with predictions) for a staged regression
A20, E4	23, 04	Remove ten unjustified `useCallback` / `useMemo` from a sample codebase with benchmarks proving parity

8.3 Retake policy

- **Category retake:** if only one category failed its floor, retake the relevant parts after at least **3 days**, using fresh variants: change the data sizes, swap the failing mechanism (e.g. context fan-out instead of unstable props), and build new traces or fixtures rather than rereading these items.
- **Full retake:** after at least **7 days** if overall < 120.
- **Do not memorise the key.** The goal is to predict and diagnose in unfamiliar code. If a retake succeeds only because items were remembered, treat it as a failed attempt.

8.4 What this assessment does not cover

- Framework-specific performance (SSR/streaming, RSC payloads, route-level data APIs): Phase 9.

- The React Compiler in depth (only its effect on the memoisation calculus is assessed, and only with the expectation that you check current docs): Phase 9 — React Compiler.
- Observability platforms and incident management at organisational scale: Phase 10.
- Performance budgets across many teams as an architecture topic: Phase 11 — Performance budgets at scale.

Next: Phase 8 — Test strategy and the pyramid, after completing the Phase 7 projects. Performance regressions you detected here become test and budget candidates there.